

SYNTAX ANALYSIS

Review of last lecture

- Implement lexical Analyzer Generator
- Follow steps:

Define patterns for your language → Define Regular Expressions for patterns → Construct transition diagram → Final states will recognize some tokens

- Unidentified patterns reported as errors
- Unidentified errors (not identified by lexer as per pattern matching) are left for detection in later stages

Agenda

- CFG
- Derivations
- Ambiguity

- For smaller programming assignments or subset of small languages, patterns can be defined to be recognized as tokens.
- For powerful languages like C, C++, JAVA there is a need to define grammar to check syntax

SYNTAX ANALYSIS: Requirement Specification

- **Input** – Program as a **token stream**
- **Output - Parse Tree** (or Abstract Syntax Tree)
- **Side-Effect – Symbol Table**
- Error - Syntax Error - Program not according to grammar (note that LA checks in accordance with patterns)
- Eg any Invalid Statement
(as per stmt defined in grammar rule)
- E.g. Function body not closed
- ... fun() ret integer { return 0; endmodule

Syntax Analyzer

- *Syntax Analyzer* creates the syntactic structure of the given source program.
- This syntactic structure is mostly a *parse tree*.
- Syntax Analyzer is also known as *parser*.
- The syntax of a programming is described by a *context-free grammar (CFG)*.

Why use context free grammars for defining PL syntax?

- Captures program structure (hierarchy)
- Employ formal theory results
- Automatically create “efficient” parsers

Context-Free Grammars

- In a context-free grammar, we have:
 - A finite set of terminals (in our case, this will be the set of tokens)
 - A finite set of non-terminals (syntactic-variables)
 - A finite set of productions rules in the following form
 - $A \rightarrow \alpha$ where A is a non-terminal and α is a string of terminals and non-terminals (including the empty string)
 - A start symbol (one of the non-terminal symbol)

Notational Conventions

1. Terminals are: lowercase letters, operator symbols, punctuation symbols, digits
2. Non Terminals : Uppercase letters, Start Symbol, lowercase italicized syntactic variables

CFG - Terminology

- $L(G)$ is *the language* of G (the language generated by G) which is a set of sentences.
 - $L(G) = \{a, aa, aaa, aaaa, \dots\}$
 - $G = \{ S \rightarrow aS \mid a \}$
- A *sentence* of $L(G)$ is a string of terminal symbols of G .
- If S is the start symbol of G then
 ω is a sentence of $L(G)$ iff $S \Rightarrow \omega$
where ω is a string of terminals of G .

- $S \Rightarrow \alpha$
 - If α contains any non-terminals, it is called as a *sentential* form of G.
 - If α does not contain non-terminals, it is called as a *sentence* of G.

Example:

1. $E \rightarrow E + E \mid E - E \mid E * E \mid E / E \mid - E$

$E \rightarrow (E)$

$E \rightarrow \text{id}$

Derivations

- This is how a grammar generates strings:
 - think of grammar rules (called productions) as rewrite rules
- Derivation: the process of generating a string
 1. begin with the start non-terminal
 2. rewrite the non-terminal with some of its productions
 3. select a non-terminal in your current string
 - i. if no non-terminal left, done.
 - ii. otherwise go to step 2.

Deriving string id+id

- At each derivation step, we can choose any of the non-terminal in the sentential form of G for the replacement.

$$E \Rightarrow (E) \Rightarrow (E+E) \Rightarrow (id+E) \Rightarrow (id+id)$$

OR

$$E \Rightarrow (E) \Rightarrow (E+E) \Rightarrow (E+id) \Rightarrow (id+id)$$

$\begin{array}{l} E \rightarrow id \mid (E) \mid E+E \\ \quad \mid E^*E \end{array}$
--

- Left-most derivation : Always expand the left-most non-terminal in each derivation step.
- what's right most derivation ?.*

Derivations($\text{deriving } \neg(\text{id}+\text{id})$)

Left-Most Derivation

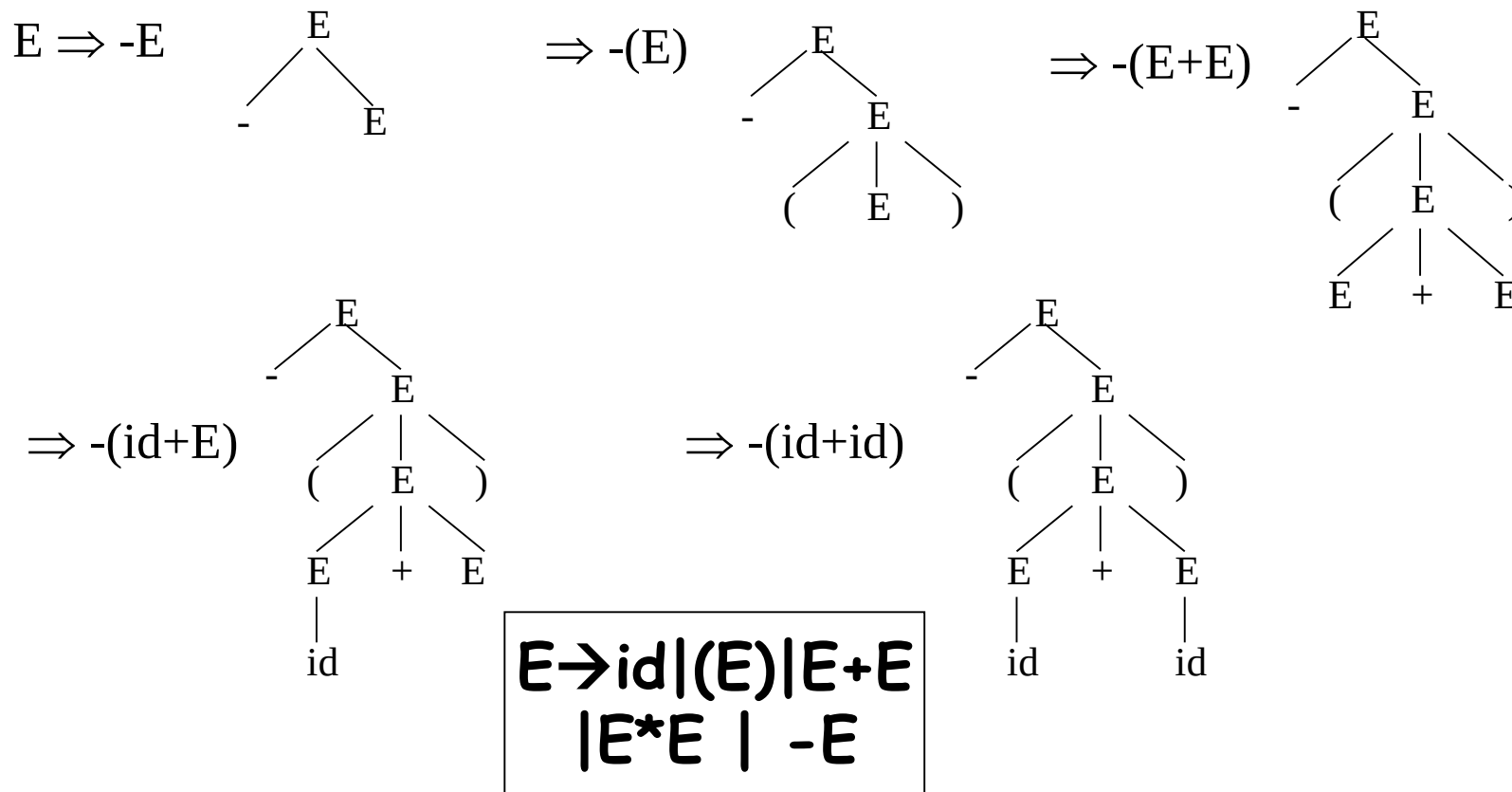
$$\begin{array}{ccccccc} \text{lm} & & \text{lm} & & \text{lm} & & \text{lm} \\ E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E+E) \Rightarrow -(id+E) \Rightarrow - \\ (id+id) & & & & & & E \Rightarrow id \mid (E \end{array}$$
$$E \rightarrow id \mid (E) \mid E + E \mid E * E \mid -E$$

Right-Most Derivation

$$E \xRightarrow{\text{rm}} -E \xRightarrow{\text{rm}} -(E) \xRightarrow{\text{rm}} -(E+E) \xRightarrow{\text{rm}} -(E+\text{id}) \xRightarrow{\text{rm}} -(\text{id}+\text{id})$$

Parse Tree

- Inner nodes of a parse tree are non-terminal symbols.
- The leaves of a parse tree are terminal symbols.



Formal Definition of a parse Tree

- Parse tree shows the derivation of a string using a grammar.
- Properties of a parse tree:
 - The root is labeled by the start symbol;
 - Each leaf is labeled by a terminal
 - Each interior node is labeled by a nonterminal;
 - If A is the nonterminal node and $X_1, \dots, X_n$ are the children nodes of A , then $A \rightarrow X_1 \dots X_n$ is a production.

Ambiguity of Grammar

- What is ambiguous grammar;
- How to remove ambiguity;

Drawbacks of Ambiguous Grammars

- Ambiguous semantics
- Parsing complexity
- May affect other phases

Example of ambiguous grammar

- Consider a grammar for expressions

$\text{expr} \rightarrow \text{expr} + \text{expr} \mid \text{expr} * \text{expr} \mid \text{digit}$

- Derivations of $9+5*2$

expr

$\Rightarrow \text{expr} + \text{expr}$

$\Rightarrow \text{expr} + \text{expr} * \text{expr}$

$\Rightarrow 9+5*2$

expr

$\Rightarrow \text{expr} * \text{expr}$

$\Rightarrow \text{expr} + \text{expr} * \text{expr}$

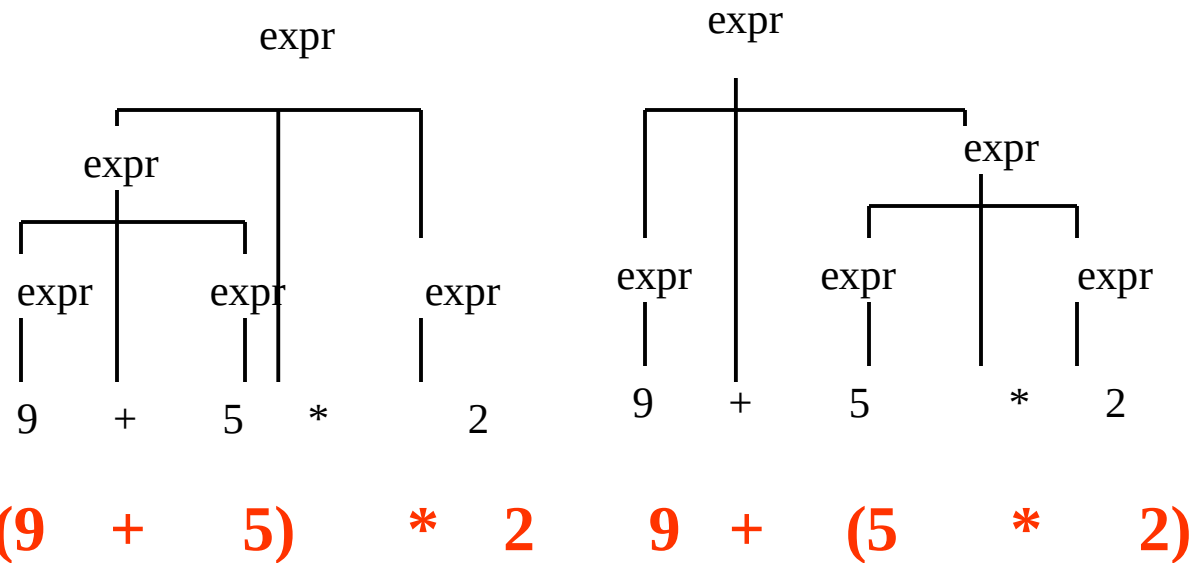
$\Rightarrow 9+5*2$

- there are different derivations**

Ambiguity of a grammar

- Ambiguous grammar: produce more than one parse tree

$\text{expr} \rightarrow \text{expr} + \text{expr} \mid \text{exp} * \text{expr} \mid \text{digit}$



– one sentence has different interpretations

Several derivations can not decide whether the grammar is ambiguous

- using the expr grammar, 3+4 has many derivations

EXPR

$\Rightarrow \text{Expr} + \text{Expr} \Rightarrow \text{Digit} + \text{Expr}$

$\Rightarrow 3 + \text{Expr} \Rightarrow 3 + \text{Digit} \Rightarrow 3 + 4$

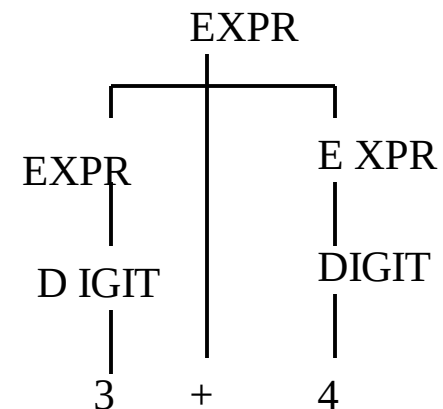
Expr

$\Rightarrow \text{Expr} + \text{Expr} \Rightarrow \text{Expr} + \text{Digit}$

$\Rightarrow \text{Expr} + 4$

$\Rightarrow \text{Digit} + 4$

$\Rightarrow 3 + 4$



- The existence of more than one parse tree deduce that the grammar is ambiguous.

Remove ambiguity

- Is there an algorithm to remove the ambiguity in CFG?
 - the answer is no
- Is there an algorithm to tell us whether a CFG is ambiguous?
 - The answer is also no.

How to remove ambiguity ?

- In practice, there are well-known techniques to remove ambiguity
- Two causes of the ambiguity in the expr grammar
 - the precedence of operator is not respected. “*” should be grouped before “+”;
 - a sequence of identical operator can be grouped either from left or from right. $3+4+5$ can be grouped either as $(3+4)+5$ or $3+(4+5)$.

- We should eliminate the ambiguity in the grammar during the design phase of the compiler.
- An unambiguous grammar should be written to eliminate the ambiguity.
- We have to prefer one of the parse trees of a sentence (generated by an ambiguous grammar) to disambiguate that grammar to restrict to this choice.

Exercise

- Consider Grammar

$$S \rightarrow aSbS \mid bSaS \mid \varepsilon$$

Derive string abab ; construct parse tree and find out whether the grammar is ambiguous or not

- How can we rewrite a grammar to incorporate associativity and precedence rules into the grammar itself?
- Reference
- Aho,Ullman Chapter 4